

TECHNICAL SOURCE AUDIT

AsymptoticInverse

Capabilities, correctness boundaries, performance, and future development

A comparison with built-in Wolfram Language

Repository	VladimirReshetnikov/Asymptotic
Paclet version	1.8.0
Audit date	9 September 2026
Commit	07a9781212beb2eeb9ff16aa625b50ac27974078
Commit timestamp	9 September 2026, 19:01:30 UTC

Central conclusion. The repository is a substantial, useful extension of Wolfram Language: its strongest contributions are explicit generalized-weight arithmetic, complete logarithmic coefficient blocks, specialized inverse coordinates, and retained error and branch information. It should be developed as a precision-aware layer *over* the native system, not positioned as a replacement for native asymptotics.

The most actionable findings concern eager dense conversion, unnecessary exact polynomial expansion, multi-index enumeration on nominally alternative inversion routes, inconsistent realness checks, and incomplete precision planning during refinement. A documented lossy native-series bridge also deserves a safer default.

Included evidence. Twenty-five independent Python tests passed, including 100 deterministic randomized truncation subcases. Native Wolfram regression probes and a proposed source-hardening script are supplied. **The Wolfram package was not executed in this audit:** the available Wolfram service failed to connect. Source-derived predictions are not presented as observed native outputs.

Abstract

This article reviews the pinned 1.8.0 implementation of the Asymptotic repository, compares its public contract with current official Wolfram documentation, and develops concrete, reproducible improvement proposals. The review examines the ordinary expansion and inversion core, series operations and refinement, arithmetic dispatch, native special-function import, branch handling, interval certificates, incremental inversion, and the Gamma/Barnes inverse constructor, together with public documentation and validation records. It identifies several source-level resource and contract problems without conflating them with numerical failures that were not observed. Two particularly clear examples are a sparse input that triggers an arbitrarily large temporary dense coefficient array, and a Newton request whose mandatory multi-index preprocessing exceeds 20,000 entries although only 142 distinct retained weights exist. Independent exact Python models verify the combinatorial counts, coefficient formulas, and a cutoff-aware powering alternative. The proposed development program prioritizes trustworthy domain and precision contracts, method-specific work planning, bounded interoperability, native regression gates, and explicit capability discovery before wider transseries generalization.

Contents

1	Executive assessment	3
2	Scope, provenance, and limits of the review	4
2.1	A fixed snapshot, not an unstable branch name	4
2.2	Evidence classes	4
2.3	What was deliberately not inferred	5
3	Architecture and implemented capabilities	5
3.1	The ordinary core	5
3.2	Operations, provenance, and exact carriers	5
3.3	Specialized coordinates and functions	5
3.4	Distribution and documentation	6
4	Comparison with built-in Wolfram Language	6
4.1	A fair capability matrix	6
4.2	Precision requests are not interchangeable	8
4.3	Recommended native baselines	8
4.4	Where each tool is the natural first choice	8
5	Mathematical invariants that the implementation must preserve	9
5.1	Blocks, order, and local finiteness	9
5.2	Product and nonlinear precision transport	9
5.3	Why derivative contracts are necessary	10
5.4	Residuals, asymptotic theorems, and certificates	10
6	Detailed findings and corrective designs	10
6.1	F01: Eager dense conversion defeats a sparse representation	10

6.2	F02: A small requested order can trigger complete polynomial powering	11
6.3	F03: Alternative inversion methods inherit the Lagrange index budget	13
6.4	F04: The real-only contract is not enforced uniformly	14
6.5	F05: Recipe refinement uses an insufficient fixed guard margin	15
6.6	F06: The native bridge loses a logarithmic remainder degree	16
7	Positive findings that should constrain the fixes	16
7.1	Exact interval certificates are more than numerical spot checks	16
7.2	Native imports preserve uncertainty through expression trees	17
7.3	The Lagrange formula and complete-block logic are sound foundations	17
7.4	Several tempting criticisms would be inaccurate	18
8	API, user experience, and maintainability	18
8.1	One result head needs a discoverable capability contract	18
8.2	Cutoffs, term goals, and the inverse observable	18
8.3	Assumptions should be explicit and stable	19
8.4	Formatting, evaluation, and automatic arithmetic	19
8.5	Migration, packaging, and diagnostics	20
9	Performance engineering and meaningful benchmarks	20
9.1	Measure the work that actually determines cost	20
9.2	Method-specific planning before low-level optimization	20
9.3	What the published benchmarks actually establish	21
10	Testing and release assurance	22
10.1	Existing coverage is substantive but scoped	22
10.2	A layered test strategy	22
10.3	The supplied tests and their precise status	23
11	Development opportunities beyond the immediate fixes	23
11.1	Backward precision planning as a reusable subsystem	23
11.2	Integration and richer logarithmic scales	23
11.3	Certificates for special functions and uncertain models	24
11.4	Systems and complex sectors are separate architectural projects	24
12	Prioritized roadmap and acceptance criteria	24
13	Reproducing and using the accompanying artifacts	25
14	Conclusion	26

1 Executive assessment

The package has grown well beyond a response to an isolated inverse-function question. Its public result head is now `GeneralizedSeries`; the canonical implementation is modular, and a generated standalone file supports single-file loading. The current metadata requires Wolfram Language 15.0 or later. Ordinary generalized power–logarithmic expansions coexist with exact exponential carriers, logarithmic coordinates, finite flat sectors, Fourier-valued coefficients, and selected special-function adapters. These are meaningful implemented capabilities, not merely an aspirational feature list.[1, 2, 3]

The right strategic distinction is *semantic control*, not a claim that Wolfram Language only knows Taylor series. Native `Series`, `InverseSeries`, and `AsymptoticSolve` already cover substantial expansion, reversion, and implicit-equation territory. The repository adds an inspectable representation and a specialized calculus that can preserve information inconvenient to express in one native series object. Conversely, native functionality remains broader in complex and multivariate problems and in integration, summation, and differential-equation asymptotics.[19, 21, 24, 23]

The following six findings have the clearest combination of source evidence and useful corrective action. “High” means important for resource safety, reliable availability, or the advertised contract; it is not a claim of exploitable remote-code execution.

ID	Priority	Finding	Recommended response
F01	High	Eager dense native-series allocation	Make conversion optional or lazy, omit implicit trailing zeros, and guard the actual dense span before allocation.
F02	High	Low-order integer powers attempt a full exact expansion	Bound speculative exact work and implement cutoff-aware integer powering after valuation normalization.
F03	High	Newton/grouped routes still enumerate the full multi-index region	Separate solver planning from Lagrange enumeration; construct a frontier by the selected method.
F04	High contract	Real-coefficient checks are inconsistent between forward and inverse paths	Establish source-domain obligations centrally and validate generated coefficients before advertising a real result.
F05	Medium	Fixed-margin refinement can miss the requested precision	Propagate required precision backward through operations, and report whether the requested goal was reached.
F06	Medium	Native <code>SeriesData</code> drops logarithmic-tail metadata	Treat this documented loss as an explicit conversion policy, not an unqualified safe bridge.

F01–F03 are algorithmic/control-flow findings. F04 is a contract gap: the resulting expression can be a perfectly valid *complex* expansion while still violating an intended real-only interface. F05 does not manufacture false coefficients; it can return a conservative bound weaker than the requested goal. F06 is *already acknowledged in the user guide*; the recommendation is to improve the safety of the interface, not to portray the caveat as undisclosed. Detailed evidence, limitations, and reproductions appear in Section 6.

The repository also deserves substantial credit. It refuses to differentiate a bare magnitude remainder without a derivative contract, distinguishes model residual checks from root certificates, retains separate absolute error scales for composite carriers, and documents the scope of its validation runs. Those choices should survive any performance refactoring.[5, 7, 4, 8, 12]

2 Scope, provenance, and limits of the review

2.1 A fixed snapshot, not an unstable branch name

The review is pinned to commit `07a9781212beb2eeb9ff16aa625b50ac27974078`, whose commit message describes the result-head rename and series-operation simplification. The packet identifies itself as version 1.8.0. All source conclusions in this article refer to that snapshot. A later `main` may already differ; reproductions should use the pinned distribution or a checkout at that commit.[\[18, 2\]](#)

The source tree contains 38 canonical kernel `.wl` files, in addition to the small initialization entry point. The repository-root standalone file is generated, rather than a second independently maintained implementation. Its inspected header records the main kernel’s normalized SHA-256, and the distribution has Git blob identifier `65363f73158c3d01e10fa274385341f9585c1efc`.[\[13, 14\]](#)

The accompanying `results/source_manifest.json` records file paths, available Git blob identifiers, review windows, and the intended depth of inspection. The main core and explicit series-operations implementation were read through overlapping windows, as were the structured native import and shared Gamma/Barnes inverse constructor. Other substantial reads covered arithmetic dispatch, forward branch orchestration, interval-certificate logic, and incremental/Newton state management. The mathematical article’s abstract and section organization were inspected, but not every included theorem chapter. Specialized flat, Fourier, and logarithmic families are discussed primarily from the current guide, loaded-module architecture, and current validation record rather than represented as fully line-by-line audited subsystems.

2.2 Evidence classes

Four distinctions are used throughout:

Source-confirmed.

A property follows directly from inspected code: for example, a `Table` dimension, an unconditional call to `indexRegion`, or the absence of a check along a traced path.

Independently checked.

A mathematical identity, resource count, or proposed restricted algorithm was executed in the supplied Python model. This is not evidence that the Wolfram implementation passed a test.

Native probe supplied.

A runnable Wolfram expression records a source-derived prediction or a proposed desired contract. Its actual result is not claimed here.

Upstream-reported.

A result appears in the repository’s validation record. It is useful historical evidence with a stated scope, not an independent rerun by this audit.

The available Wolfram service could not establish a context connection, and the subsequent version/evaluator request returned an upstream HTTP 502 error. No native kernel version was obtained, no native timings were measured, and neither the complete upstream suite nor the new Wolfram regressions ran. Direct checkout/download access from the local runtime also failed; pinned GitHub connector reads supplied the inspected source. Consequently, the archive does not pretend to contain a fully downloaded repository snapshot.

What *did* run locally is precise: 20 tests of independent mathematical/resource models and five fixture-level tests of the proposed source-edit script, all passing. One of those tests contains 100 deterministic randomized comparisons between complete and truncated rational-weight polynomial powers. Test logs and generated JSON evidence are included. A passing fixture test of an edit script is not a compiled or runtime-tested patch.

2.3 What was deliberately not inferred

The review does not infer generic native failures from the age of the original motivating questions. It does not treat a native `Series` fallback as an implementation defect by itself. It does not equate a theorem in the mathematical article with formal verification of every implementation path. It does not call an asymptotic $\mathcal{O}$ descriptor a pointwise interval bound. It also does not infer a missing feature from an old implementation plan: the development index explicitly marks historical engineering documents as historical and points readers to the current guide.[13]

3 Architecture and implemented capabilities

3.1 The ordinary core

The ordinary engine normalizes an approach to a positive local coordinate $u \rightarrow 0^+$. At a finite endpoint it uses $u = x - x_0$ or $u = x_0 - x$; at positive and negative infinity it uses $u = 1/x$ and $u = -1/x$, respectively. This makes real powers unambiguous once the source side and coefficient signs are established. The same coordinate is used to express error orders.[4, 3]

A core jet is a finite collection of pairs (β, Q_β) , where β is an exact real weight and Q_β is a polynomial in $L = \log u$. Coefficients with equal weights are merged into *complete blocks*. Ordinary exponent cutoffs retain $\beta < h$, not $\beta \leq h$. This differs from the familiar “through degree n ” reading of a Taylor-series request; the difference is material at every comparison boundary.

The inverse model has the normalized form

$$f(u) = y_0 + au^p \left(1 + \sum_{j=1}^m u^{\delta_j} Q_j(\log u) \right), \quad \delta_j > 0, \quad a \neq 0, \quad p \neq 0. \quad (1)$$

The implementation has ordinary Lagrange, grouped-Lagrange, and Newton routes; symbolic exponents are admitted through a separate depth-truncation mode under assumptions, rather than by pretending that an undecidable numeric ordering has been resolved. Individual inverse coefficients can be requested from a model.[4]

3.2 Operations, provenance, and exact carriers

The public calculus includes normalization, arithmetic, powers, logarithms, exponentials, observables, composition, truncation, refinement, and differentiation. A useful internal form is

$$s(x) = b(x) + A(x)(J(w(x)) + R(w(x))), \quad (2)$$

where b and A are exact and $w \rightarrow 0^+$. Exact carriers avoid expanding away a dominant exponential or special-function growth factor. Some compatible combinations can be flattened into a single power–logarithmic chart; otherwise the arithmetic layer can retain a composite expression with separate absolute error scales.[5, 6, 8]

Refinement may reuse stored inversion state or replay a retained source or operation recipe. Importantly, the implementation does not uniformly claim reuse: replay paths explicitly report that no coefficient reuse or work count is established. Incremental ordinary inversion uses returned-by-value state, including bounded polynomial-power caches; failed refinement is not designed to mutate the original object’s state.[5, 9]

3.3 Specialized coordinates and functions

The current guide exposes logarithmic-hierarchy, exact-core, finite-flat-sector, Fourier-coefficient, and selected special-inverse constructors. They do not amount to a universal transseries field with

unrestricted composition. The distinction matters because a single result head can suggest greater closure than any particular scale family actually provides.[3]

The Gamma/Barnes inverse constructor is a particularly substantive extension. For a large normalized logarithmic target Y , it retains the exact cores

$$X_\Gamma = \frac{Y}{W(Y/e)}, \quad X_G = \sqrt{\frac{4Y}{W(4Y/e^3)}}. \quad (3)$$

The first solves $X \log X - X = Y$. The second solves the leading quadratic-logarithmic Barnes balance. Corrections are complete polynomials in $1/\log X_\Gamma$ or $1/(\log X_G - 1)$ at successive powers of $1/X$. Affine source/target transformations and supported powers are handled in this shared constructor. Its metadata explicitly identifies finite Stirling/Barnes Poincaré reasoning and does not assert convergence or pointwise certification.[11, 29, 28]

These cores illustrate a good architectural choice: do not expand a useful exact inverse merely because a fully logarithmic expansion is possible. Retaining it can keep dominant balance, branch choice, and correction ordering much clearer. Equation (3) is not a claim that every Gamma- or Barnes-containing equation is recognized; the parser accepts a specific family of shifted/scaled single-function forms.[11]

The broad native special-function path has a different role. It proves a real source domain, normalizes selected identities, calls native `Series` with `Analytic -> False`, imports the resulting structured tree without simply discarding its remainder, and separates exponential carriers and oscillatory amplitudes. This is best understood as a carefully controlled *adapter to native algorithms*, not an independent implementation of all Bessel, Airy, hypergeometric, or elliptic asymptotics.[8]

3.4 Distribution and documentation

The modular kernel is the authoritative source. The standalone builder preserves module order and context transitions, records normalized source hashes, validates dependencies, and has a byte-for-byte freshness mode. The current workflow checks this generated-file invariant and its Python builder tests. The guide documents the renamed result head, the distinction between `Normal[s]` and the uncertainty-carrying object, and a download-then-`Get` loading procedure motivated by recorded Wolfram 15.0.1 HTTP-reader behavior. These are genuine usability and provenance improvements.[13, 15, 3]

The paclet metadata registers a kernel extension but no Documentation extension. Thus the substantial Markdown/HTML guide and mathematical article should not be confused with an already integrated native Documentation Center symbol collection. Adding searchable native reference pages, runnable examples, and paclet deployment metadata would improve discovery without changing the mathematics.[2]

4 Comparison with built-in Wolfram Language

4.1 A fair capability matrix

The comparison below is based on current official documentation and inspected repository code, not a benchmark asserting that one system wins every listed example.[19, 20, 21, 22, 24, 23]

Task	Native Wolfram Language	Repository contribution / boundary
Taylor, Laurent, Puiseux	Series and SeriesData ; existing arithmetic and calculus	Overlapping functionality; use native machinery rather than duplicate routine cases unnecessarily.
Reversion	InverseSeries accepts suitable series structures, including ramified examples	Selected real source side, explicit model, real generalized gaps, complete logarithmic blocks, and individual coefficients.
Implicit equations	AsymptoticSolve supports systems, complex or real domains, and inferred nonstandard scales	Specialized scalar inverse engines with inspectable state and error transport; not a general systems solver.
Generalized weights	A single SeriesData uses a rational lattice; surrounding expressions can carry other factors	One ordered sparse representation can combine exact real weights without forcing a common rational denominator.
Logarithmic coefficients	Native series can contain slowly varying coefficients	Explicit complete polynomial blocks and a separately recorded logarithmic degree in remainder bounds.
Composition	ComposeSeries and substitution into native series, with matching endpoint requirements	Extra branch, coordinate, and precision conditions; not every specialized representation supports generic composition.
Special functions	Extensive built-in expansion machinery	Real-domain adapters, retained carriers, conservative imported errors, and specialized Gamma/Barnes inverse coordinates.
Parameter assumptions	Native options include parameter assumptions and, in solving, generated conditions	Explicit retained conditions and failures on unproved branches; defaults and coverage differ by entry point.
Error comparison	AsymptoticLess and AsymptoticEquivalent express asymptotic relations	Object-level error transport, derivative contracts, refinement recipes, and selected exact interval certificates.
Broader asymptotics	Native integration, sums, transforms, and differential-equation interfaces	Not the package's general-purpose remit; promising future interoperability rather than an existing replacement.

Two corrections to common comparisons are essential. First, **InverseSeries** is not limited to a nonzero linear term: the official reference includes inverses of series beginning quadratically. Second, inability to store an incommensurate support in *one* rational-lattice **SeriesData** is not equivalent to inability of the whole Wolfram expression language to represent that expansion. Native symbolic factors and combinations of structured series extend beyond that one data structure.[\[21, 20\]](#)

4.2 Precision requests are not interchangeable

For the ordinary package constructor,

```
AsymptoticExpansion[f, {x, 0, 5}]
```

retains local weights strictly less than 5. A native Taylor baseline usually needs a request through degree 4 to compare the same polynomial part. Even that translation is not universal: fractional powers, cancellations, exact carriers, and term goals can change what the two outputs mean. A benchmark should compare *attained error classes and branch*, not copy the same integer into different APIs and compare elapsed time.

The package’s own meanings also vary intentionally. Ordinary cutoffs concern the positive local coordinate; Gamma/Barnes and exponential factorizations can use the bracket inside a carrier; native special-function term goals apply separately to carriers; depth truncation counts perturbation depth; and some composite objects have no single scalar cutoff. These conventions are documented, but they should be mechanically discoverable as well as explained in prose.[3, 8, 11]

4.3 Recommended native baselines

For a regular local inverse, use both a structured-series and an implicit-equation baseline. The following are supplied as reproducible native probes, not as reported outputs from this audit:

```
Clear[x, y];
InverseSeries[Series[x + x^2, {x, 0, 4}], y]
AsymptoticSolve[x + x^2 == y, {x, 0}, {y, 0, 4}, Reals]
AsymptoticInverse[x + x^2, {x, 0}, {y, 5}]
```

The independently known common polynomial is

$$y - y^2 + 2y^3 - 5y^4. \quad (4)$$

For an irrational-gap or logarithmic example, the correct question is whether the current native route returns the desired branch and order, not whether an old forum discussion said a related expression failed. For a special-function forward expansion, compare against the *same native backend* that the adapter uses; otherwise measured time includes different tasks and different guarantees.

Native asymptotic comparison predicates can be useful independent diagnostics, provided directions and parameter conditions match. They should not be reduced to a general sign comparator for oscillatory coefficients: a coefficient with infinitely many zeros may be bounded without being bounded away from zero.[25, 26]

4.4 Where each tool is the natural first choice

For routine analytic or Puiseux manipulations already well expressed as native series, native functions are a sensible baseline. For a research workflow that needs complete generalized-weight blocks, explicit transport of unknown input order, reusable inverse-model state, or a retained exact inverse core, the repository is valuable. For complex-sector, systems, or broad asymptotic integration/ODE tasks, the native system has documented scope beyond this package. This is a division of labor, not an unsupported universal performance ranking.

5 Mathematical invariants that the implementation must preserve

5.1 Blocks, order, and local finiteness

Write an ordinary object schematically as

$$J(u) = \sum_{\beta < h} u^\beta Q_\beta(\log u) + \mathcal{O}\left(u^P(1 + |\log u|)^D\right), \quad u \rightarrow 0^+. \quad (5)$$

The coefficient at a weight is the entire polynomial Q_β , not one displayed logarithmic monomial. This is essential when distinct multi-indices collide at the same weight and their leading logarithmic terms cancel. Sorting or truncating contributions before forming a complete block can give the wrong first omitted coefficient.

For finitely many strictly positive gaps δ_j , only finitely many vectors $k \in \mathbb{N}^m$ satisfy $k \cdot \delta < H$. Indeed $k_j < H/\delta_j$ for each j . Thus irrational gaps are not intrinsically an obstruction to finite-order computation. The practical obstruction is the number of vectors, the number of distinct resulting weights, and the cost of exact comparison and coefficient simplification. These are different resource quantities.

An exact zero has no remainder. An empty retained jet with finite P is only an unknown quantity of the stated order. The latter cannot generally be divided into another expression or used to prove a sign. This distinction is already respected in several package operations and should remain a central invariant.[4, 5, 6]

5.2 Product and nonlinear precision transport

Let $a = A + \mathcal{O}(u^{P_a})$ and $b = B + \mathcal{O}(u^{P_b})$, with known valuations $\alpha = \text{val}(A)$ and $\beta = \text{val}(B)$; suppress logarithmic factors temporarily. Expanding the product gives

$$ab = AB + \mathcal{O}(u^{P_a+\beta}) + \mathcal{O}(u^{P_b+\alpha}) + \mathcal{O}(u^{P_a+P_b}). \quad (6)$$

For well-formed nonempty jets with $P_a > \alpha$ and $P_b > \beta$, the cross-error is no larger than the preceding terms. Therefore the usable output precision is

$$P_{ab} = \min(P_a + \beta, P_b + \alpha). \quad (7)$$

The implementation contains the corresponding forward precision calculus. The converse is the key to refinement: to *request* output precision H , ask for

$$P_a \geq H - \beta, \quad P_b \geq H - \alpha. \quad (8)$$

A fixed additive margin cannot replace these equations when valuations can be arbitrarily negative. If $a \sim cu^\alpha$ on an admitted real branch, linearization gives the necessary leading precision relations

$$P_{a^r} = P_a + (r - 1)\alpha, \quad P_{\log a} = P_a - \alpha. \quad (9)$$

These formulas describe transport where the indicated derivatives and branch are controlled; they are not permission to take fractional powers of unknown-sign remainders. Exponentiation requires vanishing *absolute* error in the exponent to justify a first-order relative correction. The package explicitly enforces such a restriction in several exponential paths.[5, 8]

For logarithmic envelopes, the exponent equations are supplemented by polynomial degrees and by sums of competing bounds. A future precision planner should preserve that richer descriptor instead of forcing every request into one real number.

5.3 Why derivative contracts are necessary

A magnitude estimate $r(u) = \mathcal{O}(u^P)$ does not imply $r'(u) = \mathcal{O}(u^{P-1})$. For example,

$$r(u) = u^P \sin(u^{-2}) \quad (10)$$

is bounded by u^P , but its derivative contains $-2u^{P-3} \cos(u^{-2})$. An implementation that differentiates a bare `0` descriptor has silently inserted an extra hypothesis.

The repository's `SeriesDifferentiate` requires sufficient `"RemainderDerivativeOrder"`; refinement also refuses to reuse a user declaration for an old remainder as proof of a stronger refined remainder. This is a sound design choice. It may surprise users accustomed to formal series arithmetic, so the error should explain how to supply or establish the needed contract rather than merely announce failure.[\[5, 27\]](#)

5.4 Residuals, asymptotic theorems, and certificates

Three statements must stay distinct. A finite model may compose to zero below a cutoff. An exact source may satisfy an asymptotic error estimate under regularity and branch hypotheses. A particular numerical target may have a unique root in a verified interval. Neither of the first two is automatically the third.

For the ordinary inverse model, let the forward uncertainty be $\mathcal{O}(u^P \Lambda^D)$, where $\Lambda = 1 + |\log u|$, and suppose $f'(u)$ has a nonzero leading term of order u^{p-1} on the branch. A mean-value argument then gives a source error of order $u^{P-p+1} \Lambda^D$. The observable u^r has error of order

$$u^{P-p+r} \Lambda^D. \quad (11)$$

After the target coordinate is converted to its positive small form, the corresponding exponent is $(P - p + r)/|p|$. This explains the observable-dependent precision cap in the core. It also explains why the guide's `"InputRemainder"` pair declares a matching first-derivative order, not merely an unspecified value error.[\[4, 3\]](#)

The ordinary residual result explicitly says that its scope is composition with the finite forward model. The interval certificate separately says it certifies the stored explicit function, not an unspecified family represented by `"InputRemainder"`. These qualifiers are important and correct.[\[4, 7\]](#)

6 Detailed findings and corrective designs

6.1 F01: Eager dense conversion defeats a sparse representation

Evidence status: Source-confirmed allocation formula; dimension calculations independently checked; native peak memory not measured.

Location. The ordinary forward and inverse constructors eagerly call `makeSeriesData` and `makeInverseSeriesData`. Their main-core implementations, around lines 646–657 and 1012–1026, form a common denominator, calculate lattice endpoints, and allocate `Table[0, {nmax - nmin}]` before filling the retained coefficients. No allocation limit is checked at that point.[\[4\]](#)

Consider the positive-side forward request

```
AsymptoticExpansion[1 + x^(1/q) + x, {x, 0, 1}]
```

with an exact integer $q > 1$. It retains only the blocks at weights 0 and $1/q$; the first omitted term is x . Nevertheless the conversion uses

$$d = q, \quad n_{\min} = 0, \quad n_{\max} = q, \quad (12)$$

and initially allocates q coefficient slots. The input and retained sparse output have constant-size support as q grows. The source therefore admits arbitrarily large temporary allocation from a tiny input, outside the otherwise visible "MaxTerms" budget.

q	Retained blocks	Current temporary slots	Necessary native prefix
101	2	101	2
997	2	997	2
100003	2	100003	2
10^9	2	10^9	2

The last row is a *dimension calculation only*; no billion-element array was allocated. Also, the returned native object may normalize away trailing zeros. Its final `ByteCount` can therefore be small even after a large transient allocation. Measuring only the returned object's size would miss the issue.

There are two separate fixes. First, native `SeriesData` need not explicitly store all the trailing zero coefficients up to its order boundary. Allocate only through the last retained weight:

$$N_{\text{stored}} = \begin{cases} 0, & \text{no retained coefficients,} \\ d \max\{\beta : \beta \text{ retained}\} - n_{\min} + 1, & \text{otherwise.} \end{cases} \quad (13)$$

Second, guard that actual dense span. Eliminating trailing zeros alone is not enough. The request for $1 + x^{1/q} + x + x^2$ with cutoff 2 retains weights 0, $1/q$, 1, so even the minimal prefix has $q + 1$ positions. A sparse generalized object should survive when an optional dense representation is inappropriate.^[20]

The supplied `proposals/apply_bridge_hardening.py` implements both the prefix reduction and a conservative span guard in the two inspected bridge functions. By default it prints a diff and writes nothing. It verifies the pinned main-core Git blob before applying changes, refuses unexpected source shapes, and returns `Missing["DenseSeriesDataBudget", ...]` rather than invalidating the generalized result. Its configurable script-level threshold is a transitional patch design; a stable release should expose a named conversion policy or budget through the public API.

Acceptance criterion. Tiny sparse inputs must not trigger unbounded dense allocation. Tests must cover an enormous *order gap* with a short native coefficient prefix and a genuinely enormous *internal lattice span*. Memory limits should be enforced before allocation, and the returned generalized expression and remainder must be identical whether the optional bridge succeeds or declines.

6.2 F02: A small requested order can trigger complete polynomial powering

Evidence status: Source-confirmed control flow; restricted cutoff-aware alternative independently checked; predicted native failure not executed.

Location. `forwardCore` attempts `exactJet` before ordinary finite-order expansion. `exactJet` falls back on selected "infinite series" or unsupported-input failures, but propagates resource failures. Separately, `fwdPower` immediately sends a nonnegative integer exponent to `pIntegerPower`; that

routine has no working-cutoff argument. It already uses binary powering, so merely recommending exponentiation by squaring would miss the actual problem.[4]

A representative request is

```
AsymptoticExpansion[(1 + x)^256, {x, 0, 3},
  "MaxTerms" -> 200]
```

The requested answer has only three blocks:

$$1 + 256x + 32640x^2 + \mathcal{O}(x^3). \quad (14)$$

The exact route instead grows complete polynomial supports. Squaring a 17-term polynomial requires 289 retained pair products before collisions are merged, exceeding the example’s per-product budget. The failure occurs in speculative exact work even though the requested low-order result is cheap. Increasing the public budget masks the planning defect; it does not remove it.

Proposition 6.1 (Safe coefficient truncation for an exact finite power). *Let $A(u)$ be a nonzero finite generalized polynomial with least weight α , and let $n \geq 1$ be an integer. To compute all coefficients of $A(u)^n$ of weights below h , write $A = u^\alpha B$, where B has only nonnegative weights, and perform every multiplication in B^n with exclusive cutoff $H = h - n\alpha$.*

Proof. Every product contributing to B^n is a sum of nonnegative weights. Once a partial product has weight at least H , multiplication by additional factors cannot lower it below H . Thus discarded contributions cannot affect a retained coefficient. Multiplying the result by $u^{n\alpha}$ restores the requested output weights. This argument also explains why one must normalize the valuation before applying the rule to a Laurent input. $\square$

The supplied rational-coefficient model implements this proposition. It handles rational weights, negative valuations, cancellation, and exclusive boundaries, and compares truncated results against complete powers in 100 deterministic randomized subcases. It is deliberately *not* a replacement for the full package power operation: it does not transport unknown input remainders or logarithmic coefficient degrees.

Power n	Retained coefficients	Counted pair products	Peak model support
16	3	25	3
256	3	49	3
1000000	3	151	3

These are exact work counts of the *Python prototype*, not Wolfram timings. For the millionth power, the coefficients are exactly 1, 1000000, and 499999500000.

A production fix has two parts. Speculative exact expansion needs a modest budget and a recoverable “exact probe exhausted” outcome that sends the request to a finite-order route. The finite-order integer-power route must then actually respect its cutoff, with correct transport of pre-existing remainders. Fixing only one part leaves the other path expensive. The result should record why exactness was not pursued, rather than implying that a resource-limited exact probe means no valid finite-order expansion exists.

Acceptance criterion. At a fixed small cutoff, the coefficient work for $(1 + x)^n$ should grow primarily with the requested support and $\log n$, not with n full polynomial coefficients. Negative-valuation tests, logarithmic leading blocks, empty jets, and uncertain input remainders must accompany the simple binomial case.

6.3 F03: Alternative inversion methods inherit the Lagrange index budget

Evidence status: Source-confirmed unconditional preprocessing; exact combinatorial witness independently checked.

Location. In the core constructor, explicit-cutoff requests build `indexRegion` before calling `inverseBlocks`. The method selector inside `inverseBlocks` chooses Newton or grouped Lagrange only afterward. The full region also feeds the current frontier routine. Term-goal requests first grow an incremental Lagrange state, even when the requested final method is Newton, and can subsequently recompute blocks by the chosen method.[4, 9]

The following is a particularly clean witness:

```
f = x (1 + Sum[x^(1 + j/1000), {j, 1, 24}]);
AsymptoticInverse[f, {x, 0}, {y, 5}, Method -> "Newton"]
```

Here $p = r = 1$, the gaps are $\delta_j = 1 + j/1000$, and the internal retained-weight threshold is $H = 4$. Every multi-index of total degree at most 3 is inside because

$$3(1 + 24/1000) = 3.072 < 4. \quad (15)$$

Every degree-4 multi-index is outside because its weight is at least 4.004. Each such vector is a boundary neighbor of an inside vector. Therefore the mandatory region contains

$$N_{\text{inside}} = \binom{24+3}{3} = 2925, \quad (16)$$

$$N_{\text{boundary}} = \binom{24+3}{4} = 17550, \quad (17)$$

$$N_{\text{budgeted}} = 20475 > 20000. \quad (18)$$

The default multi-index budget is exceeded before the Newton solve can provide its intended advantage.

Yet the number of *distinct retained weights* is only

$$1 + 24 + 47 + 70 = 142. \quad (19)$$

For degree $n = 1, 2, 3$, the sum of the indices j attains every integer from n through $24n$, giving $23n + 1$ weights. Different total degrees lie in disjoint bands. Constant positive model coefficients ensure that contributions within each such weight share the degree-dependent sign, so these collisions do not annihilate the whole retained block.

This demonstrates a mismatch between the chosen algorithm and its preprocessing cost. It is not evidence that Newton's iteration formula is wrong. The inspected Newton state in fact doubles its precision and checks its exact residual below every claimed cutoff, which is a valuable invariant.[9]

Recommended design. Introduce method-specific planning before materializing a multi-index region. A Newton route should compute a jet and a residual-based or otherwise justified frontier without enumerating all Lagrange marker vectors. A grouped route should aggregate by weight early. It may return a conservative valid boundary when a sharp first omitted block would require expensive extra work; the object should distinguish "RemainderBound" from "SharpFrontierVerified".

For term goals, the planner should not first pay for a complete Lagrange search and then discard that computation to run Newton. An adaptive Newton strategy can enlarge a verified cutoff until enough nonzero complete blocks are present. Its bookkeeping must still account for cancellation, unknown forward precision, and exact termination.

Acceptance criterion. The witness above should not fail merely because a different algorithm’s 20,475-vector region was imposed. A post-fix test should verify the 142 retained blocks, the returned remainder, the selected method, and a below-cutoff residual. The supplied desired regression is intentionally expected to expose the current preprocessing restriction; it is not reported as a passing upstream test.

6.4 F04: The real-only contract is not enforced uniformly

Evidence status: Source-confirmed missing validation along the ordinary forward path; elementary complex branch mathematically checked; native result remains a probe.

Location. The inverse model constructor `rowsToModel` explicitly requires real leading and correction coefficients. Arithmetic regular-operand conversion also checks real polynomial coefficients. The ordinary forward path has no equivalent uniform check before building its result. `validateInput` rejects explicit inexact/complex constants in the original input, but that is not a proof that symbolic parameters or subsequently generated coefficients are real.[4, 6]

The simplest probe avoids dependence on a special-function oracle:

```
Clear[a, x];
AsymptoticExpansion[a + x, {x, 0, 2}]
```

The forward constant path accepts the coefficient a without establishing $a \in \mathbb{R}$. The public forward wrapper checks explicit approach conditions, but does not add a general source-realness obligation for this ordinary expression. By contrast, the same unresolved realness is checked when creating an inverse model or converting a regular operand for certain arithmetic operations.

A second probe checks a complex value created *after* initial validation:

```
AsymptoticExpansion[ArcCos[2 + x], {x, 0, 3}]
```

For real $x > 0$, $2 + x > 1$, so the principal inverse cosine is not real. The generic analytic forward path obtains coefficients from native `Series` and does not uniformly require those coefficients to be real before returning a forward object. The special-function candidate filter excludes the core inverse-trigonometric heads, so this example does not automatically receive the separate native-special-function real-domain proof. The expected native output is left to the supplied probe rather than invented here.[4, 10, 8]

This is *not* an assertion that the complex Taylor coefficients are mathematically incorrect. The defect is inconsistent admission criteria and incomplete statement of obligations in a package advertised around real branches. Users can otherwise obtain an exact-looking object whose realness assumptions differ according to which constructor happened to run.

A related issue deserves a clearly separated status: generic `Series` calls in the ordinary fallback use the native default analytic treatment of unrecognized functions. A result containing formal derivatives of an unknown function is not, without regularity hypotheses, an unconditional asymptotic theorem for every possible function assigned that name. Native `Series` documents its analytic default; the wrapper should either retain that obligation, request a stricter mode, or identify the result as formal. This is a *contract-design risk*, not a demonstrated numerical counterexample to a known-function expansion.[19, 4]

Recommended fix. Separate three checks: exactness of supplied constants; realness of generated coefficient data; and eventual real-domain/regularity validity of the source. A coefficient check is an effective local guard but is not a complete domain proof, especially when all retained coefficients vanish and uncertainty remains. Promote a common obligation layer used by forward, inverse, and arithmetic routes. Possible outcomes are a proved real germ, an explicitly conditional result, or a

structured failure identifying the unproved condition. Do not silently convert a complex source by applying `Re`.

Acceptance criterion. The parameter example must require or expose $a \in \mathbb{R}$; the inverse-cosine example must either be rejected by the real interface or explicitly returned in a separately supported complex mode. The same realness policy should hold for direct construction and for equivalent arithmetic construction.

6.5 F05: Recipe refinement uses an insufficient fixed guard margin

Evidence status: Source-confirmed replay formula; precision shortfall independently derived; public native probe supplied.

Location. The generic recipe replay in `SeriesRefine` requests an operand order derived from the new target h , a fixed increment of 2, and any negative *old remainder precision*. It does not generally invert the operation's valuation-dependent precision transport. Its own comments correctly state that the final operation clips to transported precision rather than inventing coefficients.^[5]

Consider

```
s = AsymptoticExpansion[1/(1 + x), {x, 0, 3}];
t = AsymptoticExpansion[x^-10, {x, 0, 3}];
r = SeriesRefine[SeriesMultiply[s, t], 5];
```

The first operand has known valuation 0 and remainder order 3; the second is an exact monomial with valuation -10 . A replay target $5 + 2 = 7$ for the first operand yields

$$x^{-10} \left(\sum_{j=0}^6 (-x)^j + \mathcal{O}(x^7) \right) = \sum_{j=0}^6 (-1)^j x^{j-10} + \mathcal{O}(x^{-3}). \quad (20)$$

Thus a result may carry the requested cutoff 5 while its attainable remainder power is only -3 . To obtain order 5, the first operand needs order 15, as Equation (8) gives immediately.

The conservative bound is not false. The shortcoming is that an operation named refinement may underdeliver a reachable request without a first-class success/failure indicator for the precision goal. More complicated compositions and nonlinear observables can require larger, non-additive changes to operand precision. A fixed margin cannot be made universally safe merely by changing 2 to a larger constant.

Recommended fix. Attach a backward precision-transfer function to each supported operation. For a product, use Equation (8). For a power or logarithm, invert Equation (9); for composition, incorporate the leading inner valuation and the derivative order of the outer germ. Refine lazily until the transported result meets the goal or a source/regularity/resource limitation is reached.

Every request should expose its requested precision, achieved precision, and whether the precision goal was satisfied. Suggested property names are `"RequestedPrecision"`, `"AchievedPrecision"`, and `"GoalSatisfied"`. On insufficient information, return a failure with a useful partial result or explicitly mark the partial object; do not overload `"Cutoff"` to imply attained accuracy. Logarithmic envelopes, exact carriers, and derivative obligations need analogous structured comparisons.

Acceptance criterion. The example should achieve at least remainder power 5, or explicitly report an unmet goal. Tests must verify the actual remainder, not just the number of retained terms, the requested cutoff field, or successful return of the result head.

6.6 F06: The native bridge loses a logarithmic remainder degree

Evidence status: Source-confirmed and documented information loss; mathematical distinction proved; safer-default proposal.

The user guide already warns that a native `SeriesData` can hide logarithmic degree and tells users to keep the separate package remainder. This finding therefore concerns the safety and clarity of an acknowledged interoperability boundary.[3]

For

```
s = AsymptoticExpansion[1 + x Log[x], {x, 0, 1}];
{s["Remainder"], s["SeriesData"]}
```

the package remainder records $\mathcal{O}(x(1 + |\log x|))$. The bridge functions use the remainder power but do not encode its logarithmic degree in the native object. An isolated native display with an $\mathcal{O}(x)$ order boundary therefore does not carry the same explicit analytic guarantee as the original object. The issue is not that the finite approximation 1 is wrong.

Indeed,

$$\frac{|x \log x|}{x} = |\log x| \longrightarrow \infty \quad (x \rightarrow 0^+), \quad (21)$$

so one cannot reinterpret the unknown tail as a literal magnitude bound $\mathcal{O}(x)$. Native series use a formal order convention that can coexist with slowly varying coefficients; a faithful *analytic-error* export must say which information was lost.[20]

The repository’s native importer itself recognizes this distinction: when reading an unknown native series tail, it conservatively loses half a lattice step to absorb any fixed polynomial in $\log u$. It obtains a sharper boundary only from a computed omitted block, not by guessing from retained coefficients. That asymmetry makes a safe outbound policy especially desirable.[8]

A conservative export can return `Missing["LogarithmicRemainder"]`, which is the behavior proposed by the hardening script. Other legitimate policies are an explicit formal/lossy export, or a weakened analytic order. For any fixed $\varepsilon > 0$ and nonnegative integer D ,

$$u^P(1 + |\log u|)^D = \mathcal{O}(u^{P-\varepsilon}), \quad (22)$$

because $u^\varepsilon(1 + |\log u|)^D \rightarrow 0$. Such weakening should be deliberate and recorded; it is not a reason to silently downgrade all native conversions.

Acceptance criterion. Every native export must state whether it preserves the explicit remainder class. An API called “safe conversion” must either preserve it, explicitly weaken it, or decline. Documentation should distinguish a formal native order boundary from an analytic magnitude bound.

7 Positive findings that should constrain the fixes

7.1 Exact interval certificates are more than numerical spot checks

The certificate implementation uses exact rational intervals and directed dyadic rounding. Its elementary exponential and logarithmic enclosures have explicit series-tail bounds; reciprocal and branch-sensitive operations refuse unsupported intervals. It checks that the whole verification interval lies on the selected source side and satisfies retained source-domain conditions. It then encloses both the forward function and its derivative, requires separation of the derivative from zero, and establishes existence using residual containment or endpoint signs.[7]

The underlying argument is elementary but strong. If $|f'(x)| \geq \mu > 0$ on an interval I , a center $c \in I$ has residual bounded by ϵ , and $[c - \epsilon/\mu, c + \epsilon/\mu] \subseteq I$, monotonicity and continuity force a root in that bracket. The same derivative separation gives uniqueness in I . A signed derivative enclosure can sharpen the bracket once existence has been established.

For example, let $f(x) = x + x^2$, target $1/10$, $I = [1/20, 3/20]$, and $c = 1/10$. Then

$$f(c) - 1/10 = 1/100, \quad f'(I) = [11/10, 13/10]. \quad (23)$$

The radius is $1/110$, and the signed correction improves the root enclosure to

$$\left[\frac{1}{11}, \frac{6}{65} \right]. \quad (24)$$

This is an independent mathematical demonstration of the certificate mechanism, not a reported execution of `InverseCertificate`.

The returned certificate states the explicit function it certified and explicitly denies certifying an unspecified "`InputRemainder`" family. This avoids a common and serious overclaim. Future support for uncertain forward models would require a numerical enclosure of that uncertainty and its derivative, not just a big- O class with an unknown constant.[7]

7.2 Native imports preserve uncertainty through expression trees

The native importer does not merely apply `Normal` and attach a guessed error. It traverses structured sums, products, powers, exponentials, and trigonometric/hyperbolic expressions, transporting absolute majorants. It rejects unresolved native trees, requires small phase errors where appropriate, verifies realness before real projection, and requires exact source equality before accepting a finite native result with no remainder.[8]

This conservative approach can be slower or reject expressions that a user expects to work. Those are optimization and explanation opportunities, not reasons to erase the checks. In particular, an oscillatory amplitude should remain bounded data rather than be treated as a nonzero leading coefficient suitable for division. Near zeros of Bessel- or trigonometric-type amplitudes, relative-error language can fail even when an absolute envelope remains useful.

7.3 The Lagrange formula and complete-block logic are sound foundations

For the normalized model (1), set $z = ((y - y_0)/a)^{1/p}$ and let $k \in \mathbb{N}^m$, $n = |k|$, and $w = k \cdot \delta$. For $n \geq 1$, the implemented complete-block contribution to u^r/z^r is

$$C_k(L) = \frac{(-1)^n r}{p^n \prod_j k_j!} \left[\prod_{j=1}^{n-1} \left(\frac{d}{dL} + r + w + pj \right) \right] \prod_i Q_i(L)^{k_i}. \quad (25)$$

The contribution is $z^w C_k(\log z)$; the empty index contributes 1. This formula is explicitly implemented in both a reference route and an incremental route using cached polynomial products.[4, 9]

Independent tests check it against Catalan coefficients for $u + u^2$, against the first logarithmic corrections for $u + u^2(1 + \log u)$, against mixed-gap collisions, and against the leading-power-2 example $u^2(1 + u)$. For the logarithmic example it gives

$$u = y - y^2(1 + \log y) + y^3(3 + 5 \log y + 2 \log^2 y) + \dots \quad (26)$$

These checks support the algebra used in the audit; they do not prove every parser, branch, or remainder path in the package.

7.4 Several tempting criticisms would be inaccurate

The core already uses binary integer powering; its problem is the absence of cutoff planning, not the absence of squaring. Core multiplication already prunes by ordered weights and scans tied boundary degrees; recommending a full Cartesian-product rewrite would regress it. Incremental inversion already caches suitable polynomial powers with a budget that does not reduce usable index capacity. Native import already treats unknown logarithmic tails conservatively. Gamma/Barnes results already distinguish "RequestedMethod" from the actual ordered-reversion method. The current validation record already says its full suite was not run. A useful audit must preserve these facts rather than recycle obsolete criticisms.[4, 9, 8, 11, 12]

8 API, user experience, and maintainability

8.1 One result head needs a discoverable capability contract

The common `GeneralizedSeries` head is an improvement over calling every object a power-logarithmic series. Nevertheless, common syntax does not imply common algebra. Ordinary jets, inverse-logarithmic coefficient rings, flat-sector objects, and composite error envelopes support different operations. The guide explicitly says that properties vary by family; `seriesData` rejects Gamma/Barnes inverse scales for generic polynomial-log operations, while dedicated routes support a subset of operations. This is documented heterogeneity, not a claim that the corresponding special constructors are absent.[3, 5, 11]

Introduce a small stable schema that every public result supplies: schema version, family, approach, assumptions, finite expression, error contract, precision request, achieved precision, and capabilities. A capability query should answer not only “supports differentiation” but “supports differentiation up to order n under this retained derivative contract.” A composite result should say that no single exponent cutoff exists, rather than require users to discover that fact by attempting truncation.

A possible diagnostic summary is:

```
<|"Family" -> "PowerLog", "SchemaVersion" -> 1,
  "Approach" -> <|"Variable" -> x, "Point" -> 0,
    "Direction" -> "FromAbove"|>,
  "RequestedPrecision" -> <|"ExponentCutoff" -> 5|>,
  "AchievedPrecision" -> <|"Power" -> 5, "LogDegree" -> 2|>,
  "GoalSatisfied" -> True,
  "EvidenceClass" -> "AsymptoticUnderHypotheses"|>
```

This is a proposed schema, not existing package syntax. Separate exact identities, formal expansions, conditional asymptotic results, and pointwise certificates. Calling all of them “verified” would conceal materially different guarantees.

Internally, replace repeated string-family switches with a small dispatch interface for scale adapters. Each adapter can define compatibility, normalization, forward precision transfer, backward precision requirements, supported observables, and refinement. The ordinary jet implementation remains the reference adapter. This reduces the risk that a new family works in a constructor but falls through to an inappropriate generic operation.

8.2 Cutoffs, term goals, and the inverse observable

A cutoff should always be accompanied by its coordinate and scope. Here it can describe absolute ordinary weights, weights inside an exact carrier, inverse-logarithmic blocks, or family-specific sector truncation. A term goal counts complete nonzero blocks, sometimes separately by carrier;

it is not generally the number of displayed summands. These conventions are explained in the guide, but they deserve concise, automatically generated annotations near results.[3]

The inverse **"Power"** option has a further documented surprise. At a finite source endpoint, power 1 returns the source variable including its translation, while other powers describe a displacement from that endpoint. Thus applying `SeriesPower[s,2]` to an inverse tending to 3 is not generally the same task as requesting its constructor with **"Power"** $\rightarrow$ 2. The former squares x ; the latter concerns $(x - 3)^2$. Do not change this silently. Add an explicit observable syntax or an option distinguishing **"Source"** and **"Displacement"**, and retain the old convention through a documented migration period.[3, 4]

Likewise, the package accepts its stated real direction strings rather than the full native direction vocabulary. A user carrying a native `Direction  $\rightarrow$  -1` into the package may be surprised. Either normalize native real-direction aliases at the boundary or reject them with an error that names the accepted equivalent. Complex directions must remain a separate capability, not be accepted merely because a native option permits them.[4, 19, 24]

8.3 Assumptions should be explicit and stable

The constructor options inspected here default **Assumptions** to **True**, whereas native expansion documentation uses **\$Assumptions**. Some internal simplification helpers also operate without an explicitly supplied assumption argument. This combination warrants dedicated session-isolation tests: repeat the same request in a clean kernel and under unrelated ambient assumptions, and verify that stored conditions and outcomes are consistent with the documented option policy. No ambient-assumption failure was executed during this audit.[4, 19, 24]

A robust design captures the intended assumptions once at the public boundary, separates parameter assumptions from approach conditions, and passes the resulting context explicitly to comparisons and simplification. Cache keys must include that context. A coefficient simplified using $a > 0$ must not be reused under $a < 0$ or under no assumption about a . Implicitly declaring every parameter real would mask F04 rather than resolve it; a genuinely conditional result should retain the missing realness condition.

8.4 Formatting, evaluation, and automatic arithmetic

The existing read-only interpretation boxes are a sound choice: editing a displayed coefficient should not leave hidden provenance claiming the old remainder. **InputForm** preserves the complete object, while **Normal** deliberately drops uncertainty. The numerical application `s[value]` substitutes into the finite expression; it is not a pointwise error estimate or a domain certificate. These distinctions are documented and should remain explicit.[4, 3]

Provide a diagnostic evaluation operation that returns the approximate value, whether the target-domain predicate is proved, the recorded asymptotic scale, and whether a numerical error bound is actually available. Do not turn the first omitted term into an error bar. Even a perfectly valid asymptotic expansion may be inaccurate far from its limiting regime, and a remainder with an unknown constant cannot establish a decimal accuracy goal.

Automatic arithmetic is convenient, but Wolfram evaluation can simplify an expression before upvalues see the intended operation. The held **SeriesNormalize** entry is specifically designed to check reciprocal operations before cancellation. This protection should be explained with examples and retained in refactoring. Tests should cover direct arithmetic, held normalization, assigned aliases, exact zero, uncertain zero, and unsupported nested expressions separately.[6]

8.5 Migration, packaging, and diagnostics

The 1.8.0 head rename intentionally does not export the old head as an alias. The guide tells users to update explicit patterns and saved input. A versioned serialization format and a migration utility would reduce this burden, especially when computation state and recipes are embedded in saved objects. Migration should validate the schema rather than perform unrestricted textual replacement in arbitrary saved code.[3, 12]

The standalone distribution is useful, but a mutable `main` URL is not a reproducibility boundary. Offer a release artifact, commit identifier, digest, and a verified-load example together. The supplied downloader demonstrates the separation of fetching, verifying, and executing: it checks the pinned Git blob and never calls `Get`. A checksum is meaningful when its expected value is obtained from a trusted record; it is not a substitute for reviewing newly downloaded code.[14, 13]

There is also a metadata consistency issue worth resolving: the root license text calls itself “MIT No Attribution License,” while the paclet and inspected source headers identify MIT. Choose the intended license designation and make the root text, source identifiers, and package metadata agree. This is a repository-consistency recommendation, not a legal determination.[16, 2, 4]

Failures should report a stable tag, failed obligation, requested family and approach, relevant operand precision, and an actionable next step. Distinguish unsupported mathematics, unproved assumptions, insufficient input information, and resource exhaustion. Retrying with a larger term budget cannot repair a genuinely unproved branch, and increasing the requested cutoff cannot improve a result whose source remainder is unknown.

9 Performance engineering and meaningful benchmarks

9.1 Measure the work that actually determines cost

A single “`MaxTerms`” parameter currently plays several roles across the inspected implementation: retained/boundary index count, raw product-pair count, expression leaf count, or working order. The unit of the budget therefore depends on the path. F01 shows a dense allocation outside that protection; F03 shows how a multi-index budget can reject an otherwise small weight-space problem. Separate limits would make both behavior and failure messages more predictable.[4, 9, 8, 11]

Track at least the number of distinct weights, raw convolution pairs, multi-indices, logarithmic coefficient size, dense native slots, and simplification work. A total wall-clock deadline and memory bound should supplement those structural counters. The native importer has individual time constraints and bounded order-doubling loops, but several individually bounded phases do not themselves define one request-wide deadline.[8]

Coefficient growth deserves separate attention. An expression with three weights can still have enormous integer coefficients or logarithmic polynomials. Repeated `Expand`, `Together`, `Simplify`, and algebraic-number normalization can dominate sparse bookkeeping. Instrument those phases before optimizing the multiplication loop further. Delay expansion where an expression DAG or a factored coefficient can be retained without compromising zero testing or complete-block comparison. Resource limits should be checked before expensive allocation or expansion, not only afterward.

9.2 Method-specific planning before low-level optimization

F02 and F03 are planning problems. Reducing one inner-loop constant cannot repair work that should never have been scheduled. Choose between exact finite expansion, cutoff-aware polynomial arithmetic, grouped weight-space computation, and multi-index coefficient enumeration using the

requested output and input structure.

For Lagrange inversion, explicit multi-indices are valuable when the caller wants individual combinatorial contributions or the region is small. For heavily colliding rational gaps, grouped weight-space methods can be preferable. Newton’s precision-doubling structure is attractive when a residual and its derivative can be evaluated efficiently. A future `Method` $\rightarrow$ `Automatic` should expose its selected method and reason. An explicit method request should not invisibly incur a more expensive alternative’s preprocessing merely to produce a frontier.

The incremental implementation currently finds the next frontier weight by sorting boundary weights and scans a layer for equality. A priority structure keyed by canonical weights could reduce repeated work on large boundaries. However, exact equality and collision completion are nonnegotiable: processing one contribution before all equal-weight predecessors arrive would invalidate complete-block cancellation. Any priority-queue redesign must preserve that invariant and compare against the existing reference route.[9]

Caches should remain request-local or explicitly attached to immutable computation state. The existing polynomial-power cache is a good example of a bounded optimization that does not reduce the usable index region. Add canonicalization and native-import caches only with keys covering assumptions, coordinate, branch, precision, and relevant backend options. A process-global cache keyed only by a displayed expression invites stale-domain errors.[9, 8]

9.3 What the published benchmarks actually establish

The repository’s current operation benchmark compares an immutable baseline with the refactored source in fresh Wolfram 15.0.1 kernels, using one warm-up and three measured samples per fixture. The recorded outputs agree exactly. The following numbers are *upstream-reported*, not rerun here.[12]

Fixture	Before (s)	After (s)
100 translated-coordinate inversions	0.12115	0.00418
Fourier merge with repeated frequencies	0.19112	0.09396
200 $\times$ 200 Fourier product below weight 4	1.28634	0.00776
Public Fourier inverse through weight 4	0.02514	0.02325

The approximately 166 $\times$ improvement belongs to one truncated-product microbenchmark, not the public package as a whole. The public inverse fixture changes much less. The validation record itself makes that distinction, and it also discusses variability in earlier native-tail timings instead of declaring a general slowdown from a single comparison. This is the right reporting discipline.[12]

Future native comparisons should use equivalent branch choices and precision contracts, not merely the same integer order or the same number of printed terms. Report cold loading separately from warm symbolic work, include failures and timeouts, measure peak allocation rather than only returned-object size, and separate model parsing, branch proof, coefficient generation, frontier construction, native conversion, and formatting. For asymptotic approximations that use different exact cores, compare achieved error at several independently chosen targets as well as symbolic residual order. Numerical agreement remains evidence, not a certificate.

10 Testing and release assurance

10.1 Existing coverage is substantive but scoped

The current validation record reports 305 passing focused tests in 18 selected files, additional isolated standalone acceptance checks, builder tests, and documentation consistency checks. It explicitly states that the full package suite was not run for the 1.8.0 milestone. This is not a repository without tests; it is a repository whose latest published native evidence is deliberately scoped.[12]

The inspected acceptance file contains independent closed-form comparisons for transformed inverses, known-source numerical targets, negative and shifted branch cases, logarithmic remainder propagation, and composition examples where vanishing derivatives improve error order. For example, it checks that composing $1 - \cos x$ with $y^2 + \mathcal{O}(y^3)$ yields $y^4/2 + \mathcal{O}(y^5)$. These tests are valuable and should not be replaced by weaker comparisons of finite expressions alone.[17]

The inspected GitHub Actions workflow runs standalone freshness and Python builder tests, not native Wolfram mathematical tests. Packaging checks cannot detect a branch-sign error, an incorrect remainder, or a broken public upvalue. Add a native release gate using an appropriately licensed execution environment, and preserve a lightweight packaging job for every relevant pull request. The full suite is particularly important for a cross-cutting result-head rename and changes to shared arithmetic helpers.[15, 12]

10.2 A layered test strategy

Use three complementary layers. First, local algebra tests should compare sparse operations against independent exact small models. Second, integration tests should exercise complete public paths with both accepted and rejected domains, verifying expressions *and* error metadata. Third, release tests should run isolated kernels across supported versions and representative platforms, with source hashes, explicit suite selection, and failure-on-empty behavior.

Differential testing against native functions is useful on shared domains, but not sufficient. A package route that calls native **Series** and then compares itself against the same native **Series** call is not an independent oracle for coefficients. Cross-method equality is also insufficient when both methods share the same faulty parser or frontier routine. Combine exact inverses, manufactured source solutions, symbolic substitution residuals, independent coefficient formulas, and adversarial contract checks.

Useful property tests include the following small set of invariant families:

1. **Precision monotonicity and replay.** Truncation never invents information; operations agree below the weaker transported precision; refinement either meets its explicit goal or reports the shortfall. Include large negative valuations and exact carrier factors.
2. **Domain and uncertainty.** An uncertain zero is not invertible merely because its displayed expression vanishes; sign and noninteger powers require proof; branch conditions survive composition and refinement; derivative declarations for an old remainder do not automatically justify a stronger refined remainder.
3. **Complete blocks and boundaries.** Colliding multi-indices are merged before counting terms; canceled frontier coefficients do not imply exact termination; logarithmic degree is retained at tied orders; both finite sides and both infinities use the correct chart.

Parameter-boundary fixtures should include leading coefficients tending to zero, two weights becoming equal, and assumptions that do not decide an ordering. Returning a structured unproved condition is preferable to silently sorting by floating-point approximations. For native adapters, exercise both successful imports and deliberately unresolved or nonreal series trees.

For certificates, test invalid closed intervals, unsupported functions, zero-containing derivative enclosures, and the difference between an explicit source and an unknown remainder family.

10.3 The supplied tests and their precise status

This archive includes 25 executed Python tests: 20 for independent rational-weight/resource models and five for the proposed text-edit procedure. They cover the dense-size calculation, cutoff-aware powering, complete-versus-truncated agreement, mixed-gap counting, Euler coefficients, and the refinement precision calculation. The test log and JSON evidence record exactly those results.

The supplied Wolfram characterization script has 14 probes and records the actual kernel version, system ID, package hash, messages, elapsed time, and compact result data when run. It uses per-probe time and memory limits and does not download code. The separate eight-test `proposed_regressions.wlt` file expresses *desired post-fix behavior*; several tests are intentionally expected to fail on the original implementation. Neither script was executed here. Its proposed policy of rejecting unproved real coefficients can alternatively be implemented as a clearly conditional result, in which case the corresponding regression expectation should be adapted.

The source-hardening proposal addresses F01 and F06 only. It verifies the canonical core blob, checks exact edit preconditions, prints a diff by default, and requires `-write` to modify a local checkout. It was tested against small fixtures, not a complete retrieved source file. It is not a replacement package, not a tested Wolfram hotfix, and not an implementation of the F02–F05 architectural changes.

11 Development opportunities beyond the immediate fixes

11.1 Backward precision planning as a reusable subsystem

Turn the F05 repair into a general planner rather than accumulating larger guard constants. Each operation should expose a monotone map from operand precision contracts to an output contract and a conservative inverse requirement map. Products use valuations; powers use the leading valuation and exponent; analytic observables use the first relevant nonzero derivative; composition uses the inner approach rate and the outer remainder.

The planner should operate on a precision lattice, not only a scalar cutoff. Ordinary (P, D) classes, inverse-logarithmic coefficient orders, exponential carriers, and sectorwise bounds are not interchangeable. A partial order can express that one bound implies another even when no single global order ranks all sectors. The existing composite envelopes supply a starting point for preserving incomparable information rather than forcing it into one scalar.[5, 8, 6]

A replayable operation DAG can then request just enough source precision, share repeated subexpressions, and retain a reason when a source cannot provide more information. The result should distinguish “source exhausted,” “budget exhausted,” and “goal attained.” This planner would improve correctness of expectations, performance, and diagnostics simultaneously.

11.2 Integration and richer logarithmic scales

Generic integration is a natural extension of the public calculus, but its contract must address constants and the integrability of the remainder. For a polynomial logarithmic block and $\beta \neq -1$,

$$\int u^\beta (\log u)^k du = \frac{u^{\beta+1}}{\beta+1} \sum_{j=0}^k \frac{(-1)^j k!}{(k-j)!(\beta+1)^j} (\log u)^{k-j}. \quad (27)$$

At $\beta = -1$, the primitive is $(\log u)^{k+1}/(k+1)$. For a definite integral from zero, $\mathcal{O}(u^P(1 + |\log u|^D))$ with $P > -1$ integrates to the corresponding order $P+1$; other cases require a different normalization and careful constant handling. A blockwise formula alone is not an integration theorem for arbitrary remainder classes.

Rational functions of logarithms and nested logarithmic coefficients would close more useful operations than polynomial logarithms alone. The existing specialized logarithmic constructors demonstrate demand, but one should define a precise admitted hierarchy, comparison procedure, and closure theorem before merging all families into a purported universal representation. Parameter-dependent scales may require case splitting into regions with fixed leading signs and weight orderings, not one unconditional output.

11.3 Certificates for special functions and uncertain models

Extending certificates to Gamma, Barnes, Bessel, and other special functions would be valuable only with rigorously enclosed function and derivative values on the relevant real intervals. The current elementary rational evaluator should remain a small trusted component. An adapter must provide explicit tail bounds or validated interval evaluations, state its domain, and refuse unsupported parameter ranges. Native high-precision evaluation by itself is not the same guarantee.[\[7, 8, 11\]](#)

For an uncertain forward family, numerical bounds on both the value error and derivative error are required. The existing "InputRemainder" order declaration is insufficient because it does not specify a constant or a finite validity threshold. A future family certificate should explicitly identify which quantified uncertainty class it encloses and whether existence and uniqueness hold uniformly across it.

Numerical evaluation also deserves an independent layer. Retained logarithmic targets and exact Lambert cores can avoid evaluating huge original forward values, but finite-precision cancellation and target-domain checks still matter. Report precision loss and use stable normalized equations when available. Do not silently convert an approximate input into an exact rational interpretation: that changes the mathematical problem rather than simply increasing precision.

11.4 Systems and complex sectors are separate architectural projects

A multivariate implicit extension would replace the scalar derivative with a Jacobian and require an invertible leading linearization, a compatible scale algebra, and a joint precision plan. Native `AsymptoticSolve` already provides an important systems baseline, so this extension should target capabilities not already conveniently available: explicit generalized-weight coefficients, retained uncertainty, or certified branch information.[\[24\]](#)

Complex-sector expansions require more than replacing `Reals` in a predicate. They need branch conventions for logarithms and powers, sector domains, dominance relations that may change with direction, and explicit handling of exponentially small contributions and Stokes transitions. These are worthwhile research directions, but they should not weaken the currently intelligible real-branch contract. Define the smallest useful sector class first and supply examples where ordinary real or native representations are inadequate.[\[23, 27\]](#)

12 Prioritized roadmap and acceptance criteria

The following sequence is organized by dependency and value, not by a promised delivery schedule.

Stage	Work	Acceptance criterion
P0	Bounded native conversion; explicit logarithmic bridge policy; real-domain validation; precision-goal reporting; native regression gate	F01/F04/F05/F06 fixtures produce the intended result or an explicit, accurate failure/conditional status. Full native suite passes at the release commit. No hidden dense allocation escapes the stated budget.
P1	Cutoff-aware powers; method-specific inverse preprocessing and frontier computation; backward precision planner	F02 succeeds with small output work. F03 no longer enumerates an unnecessary simplex. F05 achieves the requested precision when its retained sources can supply it. Independent residual and boundary tests pass.
P2	Stable result schema, capability discovery, migration tools, integrated reference pages, reproducible release artifacts	Every family advertises its valid operations and error meaning. Saved objects carry a schema version. Pinned load examples and generated distributions agree with canonical sources.
P3	Richer logarithmic coefficients, integration, validated special-function adapters, selected systems or sector extensions	Each extension states its domain, closure and remainder rules, independent oracles, and failure boundaries before being exposed as a generic operation.

The immediate engineering objective should be a smaller gap between what the object *claims*, what the computation *actually achieved*, and what the user *requested*. Adding more special-function names before closing that gap will increase the surface area of the same problems. Conversely, a precision planner and capability schema will make later extensions cheaper and more trustworthy.

13 Reproducing and using the accompanying artifacts

The archive is self-contained for rebuilding this article and rerunning its Python checks. It is deliberately not represented as a complete upstream repository mirror. Its main files are:

Artifact	Purpose and status
<code>article.tex</code> , <code>references.tex</code> , <code>references.bib</code> , <code>article.pdf</code>	Review source, bibliography, and rendered article.
<code>code/audit_models.py</code>	Independent exact restricted models, not an alternative implementation of the complete package.
<code>code/test_audit_models.py</code> , <code>code/test_patch_proposal.py</code> <code>code/generate_evidence.py</code>	Executed Python tests.
<code>code/native_characterization.wl</code>	Regenerates exact resource and coefficient evidence and local code hashes.
<code>code/proposed_regressions.wlt</code> , <code>code/run_proposed_regressions.wl</code>	Native behavior recorder; not executed here.
<code>proposals/apply_bridge_hardening.py</code>	Desired post-fix native acceptance tests and runner; not executed here.
<code>code/download_pinned.py</code>	Dry-run-by-default, pinned-source proposal for F01/F06; fixture-tested only.
<code>results/</code>	Fetches and verifies the pinned standalone source without executing it; not executed here.
	Executed Python log, mathematical/resource evidence, source-review manifest, and rendering checks.

Run the independent checks from the archive root:

```
python -m unittest discover -s code -p 'test_*.py' -v
python code/generate_evidence.py
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

To characterize the upstream implementation, obtain the pinned standalone file or use a checkout at the stated commit, set `ASYMPTOTIC_AUDIT_PACKAGE` to the local file, and launch `code/native_characterization.wl` in a fresh Wolfram kernel. The script records the expected audit commit separately from the actual file hash; a modular entry's hash does not identify every companion file. Use a complete commit-pinned checkout for modular tests, or the verified standalone distribution. The README contains example commands for Unix-like shells and PowerShell.

Review the hardening proposal against the canonical kernel source, never the generated standalone duplicate. After an intentional application, regenerate the standalone distribution with the upstream builder and run both native acceptance and the full upstream suite. The post-fix suite is not expected to become wholly green from the bridge proposal alone; its other fixtures concern changes not implemented by that proposal.

14 Conclusion

The repository's central contribution is a usable, inspectable language for selected real generalized asymptotic expansions and inverse problems. Complete logarithmic blocks, exact inverse cores, retained branch metadata, conservative error transport, and the separation of formal residuals from rational interval certificates are worth preserving. Native Wolfram Language remains both a powerful baseline and an essential backend; the strongest product direction is complementary integration rather than a claim of wholesale replacement.

The most important immediate improvements are concrete. Stop eager dense conversion from defeating sparse computation. Avoid fully expanding large exact powers for tiny requested cutoffs. Let Newton and grouped methods avoid Lagrange's full multi-index preprocessing when it is not mathematically needed. Apply a consistent real-domain contract. Plan refinement from the required output error backward to its sources, and make lossy native conversion explicit.

The supplied proofs, exact resource models, guarded edit proposal, and native probes turn these recommendations into reviewable work items. They also leave the boundary of evidence visible: 25 independent Python tests passed; upstream validation was inspected but not rerun; native Wolfram execution was unavailable. No universal correctness proof, complete theorem audit, or package-wide speedup is asserted. The next release should earn stronger claims through a full native regression run and matched-precision benchmarks at a fixed source revision.

References

- [1] Asymptotic Contributors. *Repository overview*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/README.md>.
- [2] Asymptotic Contributors. *Paclet metadata*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/PacletInfo.wl>.
- [3] Asymptotic Contributors. *AsymptoticInverse user guide*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Documentation/UserGuide.md>.
- [4] Asymptotic Contributors. *Core generalized jets, forward expansion, inversion, and native bridges*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/AsymptoticInverse.wl>.
- [5] Asymptotic Contributors. *Explicit series operations and refinement recipes*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SeriesOperations.wl>.
- [6] Asymptotic Contributors. *Automatic arithmetic and held normalization*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/SeriesArithmetic.wl>.
- [7] Asymptotic Contributors. *Exact rational inverse certificates*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/InverseCertificates.wl>.
- [8] Asymptotic Contributors. *Structured native special-function expansion import*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/NativeSpecialFunctions.wl>.
- [9] Asymptotic Contributors. *Incremental coefficients and Newton precision doubling*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/IncrementalInverse.wl>.
- [10] Asymptotic Contributors. *Applied inverse expressions and forward branch orchestration*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/InverseFunctionExpressions.wl>.
- [11] Asymptotic Contributors. *Shared Gamma and Barnes inverse constructor*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Kernel/GammaInverse.wl>.

- [12] Asymptotic Contributors. *Review and validation record*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/validation/README.md>.
- [13] Asymptotic Contributors. *Development and provenance notes*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/docs/development/README.md>.
- [14] Asymptotic Contributors. *Generated standalone distribution*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse.wl>.
- [15] Asymptotic Contributors. *Standalone freshness workflow*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/.github/workflows/standalone.yml>.
- [16] Asymptotic Contributors. *Root license text*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/LICENSE>.
- [17] Asymptotic Contributors. *Acceptance tests for coordinates and calculus*. Pinned revision 07a9781; inspected 9 September 2026. <https://github.com/VladimirReshetnikov/Asymptotic/blob/07a9781212beb2eeb9ff16aa625b50ac27974078/AsymptoticInverse/Tests/AcceptanceCoordinatesCalculus.wlt>.
- [18] Asymptotic Contributors. *Rename the result head to GeneralizedSeries and simplify series operations*. Commit timestamp 9 September 2026, 19:01:30 UTC. <https://github.com/VladimirReshetnikov/Asymptotic/commit/07a9781212beb2eeb9ff16aa625b50ac27974078>.
- [19] Wolfram Research. *Series: Wolfram Language documentation*. Official documentation accessed 9 September 2026; capability comparison, not an executed benchmark. <https://reference.wolfram.com/language/ref/Series.html>.
- [20] Wolfram Research. *SeriesData: Wolfram Language documentation*. Official documentation accessed 9 September 2026; capability comparison, not an executed benchmark. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [21] Wolfram Research. *InverseSeries: Wolfram Language documentation*. Official documentation accessed 9 September 2026; capability comparison, not an executed benchmark. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [22] Wolfram Research. *ComposeSeries: Wolfram Language documentation*. Official documentation accessed 9 September 2026; capability comparison, not an executed benchmark. <https://reference.wolfram.com/language/ref/ComposeSeries.html>.
- [23] Wolfram Research. *Asymptotic: Wolfram Language documentation*. Official documentation accessed 9 September 2026; capability comparison, not an executed benchmark. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [24] Wolfram Research. *AsymptoticSolve: Wolfram Language documentation*. Official documentation accessed 9 September 2026; capability comparison, not an executed benchmark. <https://reference.wolfram.com/language/ref/AsymptoticSolve.html>.
- [25] Wolfram Research. *AsymptoticLess: Wolfram Language documentation*. Official documentation accessed 9 September 2026; capability comparison, not an executed benchmark. <https://reference.wolfram.com/language/ref/AsymptoticLess.html>.

- [26] Wolfram Research. *AsymptoticEquivalent: Wolfram Language documentation*. Official documentation accessed 9 September 2026; capability comparison, not an executed benchmark. <https://reference.wolfram.com/language/ref/AsymptoticEquivalent.html>.
- [27] National Institute of Standards and Technology. *Digital Library of Mathematical Functions, Section 2.1: Definitions and elementary properties of asymptotic approximations*. Accessed 9 September 2026. <https://dlmf.nist.gov/2.1>.
- [28] National Institute of Standards and Technology. *Digital Library of Mathematical Functions, Section 5.11: Gamma-function asymptotic expansions*. Accessed 9 September 2026. <https://dlmf.nist.gov/5.11>.
- [29] National Institute of Standards and Technology. *Digital Library of Mathematical Functions, Section 4.13: Lambert W-function*. Accessed 9 September 2026. <https://dlmf.nist.gov/4.13>.